Computer Architecture Lab 1 Report

Ray 2026-05-03

1 Lab Requirements 2
2 Single-Cycle CPU Architecture 2
3 Pipelined CPU Architecture 2
3.1 Baseline Pipeline 2
3.2 Hazard Resolution 3
3.3 Exceptions and Precise State 3
3.4 Modular Project Structure 3
3.5 Multi-Cycle Multiplier and Divider 4
3.5.1 Multiplier 4
3.5.2 Divider 4
3.6 Issues and Solutions 5
4 Performance Evaluation 6
4.1 Test Environment and Parameters 7
4.2 Performance Analysis 7

1 Lab Requirements

The goal of this lab is to design and implement a RISC-V CPU in SystemVerilog. The main requirements are:

1. **Baseline single-cycle CPU**: Implement a single-cycle CPU that supports the base integer instruction set.
2. **Pipeline conversion**: Convert the single-cycle design into a classic five-stage pipeline.
3. **Hazard handling**: Resolve data hazards and control hazards in the pipeline.
4. **Verification and testing**: Connect the design to the Difftest framework so the committed architectural state matches the reference simulator, then pass functional tests such as `lab1-test.bin`.

2 Single-Cycle CPU Architecture

The single-cycle CPU is the baseline for the later pipeline design. In this architecture, every instruction completes fetch, decode, execute, memory access, and write-back in one clock cycle. Although the cycles per instruction is strictly 1.0, the critical path is very long, usually running from instruction memory through the ALU and then to data memory. As a result, the maximum clock frequency is extremely low.

3 Pipelined CPU Architecture

3.1 Baseline Pipeline

This lab implements a classic five-stage pipeline: instruction fetch, decode, execute, memory access, and write-back. To match the strict timing requirements of the Difftest framework, the design also adds a complete commit-stage register after write-back. This removes the one-cycle synchronization mismatch between sequential logic, such as register-file write-back on the rising edge, and the combinational checkpoint observed by Difftest.

Pipeline registers are inserted between stages so the instruction data flow and metadata are packaged and forwarded together. The design follows a “decode once, use everywhere” approach, which keeps the control logic in later stages much simpler.

3.2 Hazard Resolution

Data hazards and forwarding To solve read-after-write hazards, the design introduces a forwarding unit. When the destination register of an instruction in the execute, memory, or write-back stage overlaps with a source register of the current decode-stage instruction, the newest value is forwarded directly to the execute stage. This avoids unnecessary pipeline stalls and improves instruction throughput.

Control hazards Branch instructions create control hazards. Because this design does not yet include a dynamic branch predictor, it uses static not-taken prediction plus flushing. When the execute stage discovers that a branch should be taken, the incorrect frontend pipeline registers are flushed and the program counter is redirected.

3.3 Exceptions and Precise State

To handle custom exit instructions, potential memory violations, and illegal instructions correctly, the CPU implements a precise exception mechanism based on a poison-bit style marker.

Illegal-instruction marking When the decoder sees an unsupported opcode, it does not halt immediately. Instead, it marks the instruction as illegal and lets that metadata travel down the pipeline.

Commit-stage filtering The exception is only raised if the marked instruction reaches the commit stage and has not been flushed by an earlier branch. This prevents wrong-path instructions fetched after a branch misprediction from reporting false errors.

With this mechanism, the project follows a construction-by-correctness style: transient frontend mistakes cannot leak into later architectural state.

3.4 Modular Project Structure

The CPU is organized as a set of loosely coupled modules. The top-level module instantiates each pipeline stage and wires the inter-stage registers. The execute stage, or EXU, is designed as a flexible dispatch layer.

Instead of handling complex hazard logic internally, the EXU receives already-selected operands from a dedicated operand selection unit. It then dispatches those operands to the ALU, the multi-cycle multiplier, or the multi-cycle divider

according to the instruction type. This separation between dispatch and execution makes it much easier to add floating-point units or custom accelerators later.

3.5 Multi-Cycle Multiplier and Divider

3.5.1 Multiplier

Principle The multiplier is based on segmented shift-and-add accumulation. A 64-bit multiplication in one cycle would create a long critical path, so the operation is split across a parameterized number of cycles. For example, when the multiplier is configured for four cycles, each cycle processes 16 bits of the multiplier and accumulates the partial product.

Implementation The module maintains shifted operand registers and an accumulator. On the first enabled cycle, it captures the operands. On later cycles, it performs `accumulator + op1 * op2[SEG-1:0]` and shifts the operands for the next segment.

```
accumulator ⩽ accumulator + op1_reg * op2_reg[SEG - 1 : 0];
op1_reg ⩽ op1_reg << SEG;
op2_reg ⩽ op2_reg >> SEG;
remain_cycles ⩽ remain_cycles - 1;
```

After the computation finishes, the module raises a done signal, handshakes with the execute stage, and releases the pipeline freeze.

```
end else if (ack) begin
  done_reg ⩽ '0;
end else if (remain_cycles ≠ '0) begin
   ...
  if (remain_cycles = 1) done_reg ⩽ 1'b1;
```

3.5.2 Divider

Principle The divider uses restoring division and a 64-cycle state machine. The core divider operates on unsigned values, so signed operands are first converted to absolute values. The state machine repeatedly shifts the

divisor, attempts a subtraction, and emits either a one bit or a zero bit in the quotient.

Implementation On the first clock cycle, combinational preprocessing converts the operands to absolute values, stores them in 128-bit shift registers, and latches the final sign information.

```
dividend     ≤ {64'b0, abs_op1};
divisor      ≤ {1'b0, abs_op2, 63'b0};
 ...
quo_neg_reg ≤ (op2 ≠ 0) && (op1_neg ^ op2_neg);
rem_neg_reg ≤ op1_neg;
```

The next 64 cycles perform the shift-subtract iteration. Afterward, post-processing restores the signs of the quotient and remainder using the sign information captured in the first cycle.

```
end else if (counter ≠ '0) begin
  if (dividend ≥ divisor) begin
    dividend ≤ dividend - divisor;
    accumulator ≤ (accumulator << 1) | 64'b1;
  end else begin
    accumulator ≤ accumulator << 1;
  end
  divisor ≤ divisor >> 1;
  counter ≤ counter - 1;
```

3.6 Issues and Solutions

Core issue	Scenario and symptom	Architectural solution
Forwarding priority	When the memory stage and write-back stage both try to forward data for the same source register, the older value can overwrite the newer value.	In the SystemVerilog combinational block, the write-back forwarding condition is checked first and the memory-stage condition is checked afterward. The physical assignment order ensures that the newest value takes priority.
Transient-data loss	A long-latency multiply or divide freezes the	The multiplier and divider latch operand metadata internally.

Core issue	Scenario and symptom	Architectural solution
	frontend, while later stages drain. Forwarded operands can disappear from the bypass network by the second cycle.	Cycle 0 permanently captures the operands and sign bits in registers, isolating the operation from later pipeline movement.
Divider operand truncation	RV64M word instructions pass 32-bit values through 64-bit physical registers. If the upper 32 bits contain stale data, the divider state machine can compute a nonsensical result.	The ALU adds a dynamic preprocessing path. It detects word operations and zero-extends or sign-extends the 32-bit operands to 64 bits before division.
Division by zero	For division by zero, the quotient should be all ones, or -1. If the ordinary signed post-processing path blindly negates that value, it can incorrectly become $+1$.	The restoring divider naturally produces all ones when the divisor is zero. The sign latch masks negation unless the divisor is nonzero and the input operands have opposite signs.

4 Performance Evaluation

The performance comparison between the single-cycle CPU and the pipelined CPU uses the following core equation:

$$\text{Execution Time} = \text{Instruction Count} \times \text{CPI} \times \left(\frac{1}{F_{\max}}\right)$$

Architecture	Difftest IPC (lab1/lab1-extra)	Critical Path Delay (ns)	Estimated Frequency (MHz)
Single Cycle	0.500 / 0.500	231.404	4.3
Pipeline (Mul Cycle=2)	0.500 / 0.038	15.849	63.1

Architecture	Difftest IPC (lab1/lab1-extra)	Critical Path Delay (ns)	Estimated Frequency (MHz)
Pipeline (Mul Cycle=4)	0.500 / 0.038	13.686	73.1
Pipeline (Mul Cycle=8)	0.500 / 0.038	13.688	73.1
Pipeline (Mul Cycle=16)	0.500 / 0.037	13.689	73.1
Pipeline (Mul Cycle=32)	0.500 / 0.035	11.044	90.5

4.1 Test Environment and Parameters

The timing results come from synthesis timing reports generated by tools such as Vivado. The critical path delay is extracted from those reports and used to estimate the maximum physical clock frequency, $F_{\max}$.

Two workloads are used for IPC measurement:

- `lab1`: Focuses on base RV64I integer operations and control flow.
- `lab1-extra`: Heavily uses RV64M multiply and divide instructions, stressing the multi-cycle execution units.

The parameter `Mul Cycle` is the number of cycles used by the multiplier state machine. A smaller cycle count means each cycle must handle a wider multiplication segment.

4.2 Performance Analysis

The data highlights four important facts about the physical implementation:

1. **Pipeline frequency improvement**: The single-cycle design must complete a full 64-bit divide in one clock cycle, effectively placing a long chain of subtractors in one critical path. This produces a 231.4 ns delay and an estimated frequency of only 4.3 MHz. After introducing the pipeline and converting multiply/divide into multi-cycle state machines, the delay drops to about 15.8 ns, producing a large frequency improvement.
2. **DSP and routing delay wall**: For `Mul Cycle` values of 4, 8, and 16, the critical path does not improve much and stays near 13.68 ns. This is not a logic bug; it reflects the FPGA implementation. The synthesis tool maps

multiplication onto fixed DSP slices, and the internal DSP delay plus routing delay becomes roughly fixed across those widths.

3. **Carry-chain limit**: When `Mul Cycle` reaches 32, each cycle only multiplies a 2-bit segment. The synthesis tool can use lighter logic instead of a large DSP path, reducing the delay to about 11.04 ns. Further improvement is limited by the 64-bit accumulator, whose carry chain becomes the new physical floor.
4. **IPC and frequency trade-off**: On the multiply/divide-heavy `lab1-extra` workload, increasing `Mul Cycle` from 2 to 32 reduces IPC from 0.038 to 0.035 because the execute stage freezes the pipeline longer. This illustrates the architectural trade-off between instruction-level parallelism and a shorter global critical path.